DIGITAL FORENSICS LOG ANALYZER

OOP Project Report

BSCYS-F-25-B

Object-Oriented Programming

Department of Cyber Security

Submitted by:

Abdullah Sajid 2500705

Hasan Ahmed 2500729

Maheera Sarfraz 2501651

Abdul Rehman 2500699

Submitted to:

Ms.Irum Andleeb

Table of Contents

1. Project Overview

The Digital Forensics Log Analyzer is a comprehensive C++ console application developed on the principles of Object-Oriented Programming course. The system provides a multi-case forensics investigation platform capable of ingesting raw system log files, detecting security threat patterns through rule-based analysis, and generating structured investigation reports.

The project demonstrates the practical application of core OOP principles like encapsulation, inheritance, polymorphism, and dynamic memory management in a real-world cybersecurity context.

1.1 Core Features

- Multi-user authentication system with role-based access control (Admin / User)
- Account management: registration, login, lockout after failed attempts, unlock
- Multi-case forensics workspace supporting up to 10 simultaneous cases
- Dual log parser hierarchy: Sample Log Parser and Real Linux Log Parser
- 8-rule threat detection engine raising CRITICAL and WARNING alerts
- On-screen and file-based investigation report generation
- Sample log file generator for demonstration and testing
- Admin control panel for user management (promote, demote, delete, unlock)

2. OOP Concepts Demonstrated

2.1 Classes and Objects

The project is built around two primary classes that model the real-world entities in a forensics investigation:

Class	Role	Key Responsibilities
Analyst	Models an investigator or admin user	Stores name, badge ID, role; provides isAdmin() check; manages totalAnalysts counter
ForensicCase	Models a forensics investigation case	Stores events and alerts dynamically; runs analysis; generates reports; manages its own memory
LogParser (Abstract)	Base class for all parsers	Defines the interface via pure virtual parseAndLoad()
SampleLogParser	Concrete parser for sample logs	Parses structured [timestamp] EVENT key=value format
RealLinuxLogParser	Concrete parser for Linux auth logs	Parses /var/log/auth.log syslog-format lines

2.2 Encapsulation

Both core classes make heavy use of encapsulation to protect internal state and expose clean interfaces.

ForensicCase Encapsulation

- Private members: caseID (const), title, events array, alerts array, capacities, event/alert counts
- Private methods: expandCapacity(), expandAlertCapacity(), extractHour() — internal helpers not exposed
- Public interface: addEvent(), addAlert(), runAnalysis(), printReport(), saveReport(), getters

Analyst Encapsulation

- Private members: name (raw char*), badgeID, role, static totalAnalysts counter
- Public interface: getName(), getBadgeID(), getRole(), isAdmin(), displayInfo()

```
class Analyst {
```

```
private:
    char* name;         // dynamically allocated
    int badgeID;
    string role;
    static int totalAnalysts;
public:
    bool isAdmin() const { return role == "admin"; }
    // ... constructors, destructor, getters
};
```

2.3 Inheritance

The log parsing subsystem uses a clean inheritance hierarchy to support multiple log formats without changing client code:

Class	Type	Method
LogParser	Abstract Base Class	virtual bool parseAndLoad(...) = 0
SampleLogParser	Concrete Derived Class	Overrides parseAndLoad() for structured sample logs
RealLinuxLogParser	Concrete Derived Class	Overrides parseAndLoad() for Linux auth.log format

```
class LogParser {
protected:
    string currentFileName;
public:
    virtual bool parseAndLoad(const string& f, ForensicCase* t) = 0;
};

class SampleLogParser : public LogParser { ... };
class RealLinuxLogParser : public LogParser { ... };
```

2.4 Polymorphism

The parser selection in the main menu demonstrates runtime polymorphism via virtual function dispatch. The client code holds a base-class pointer and calls the same method regardless of which parser was instantiated:

```
LogParser* parser = nullptr;
if (filename.find("Linux") != string::npos)
    parser = new RealLinuxLogParser();
else
    parser = new SampleLogParser();


parser->parseAndLoad(filename, cases[activeCaseIndex]);
delete parser;   // virtual destructor called correctly
```

2.5 Constructors and Destructors

The Analyst class demonstrates the full canonical form default constructor, parameterized constructor, copy constructor, and destructor all managing heap-allocated memory safely:

Constructor / Destructor	Purpose
Analyst()	Default: allocates 16 chars, sets 'Unassigned', increments counter
Analyst(const char*, int, string&)	Parameterized: allocates exact name length + 1, sets role
Analyst(const Analyst&)	Copy: deep-copies name buffer, increments counter
~Analyst()	Destructor: delete[] name, sets to nullptr, decrements counter
ForensicCase()	Default: initializes with caseID from counter, 4-element dynamic arrays
ForensicCase(const ForensicCase&)	Copy: deep-copies events and alerts arrays, appends '(Copy)' to title
~ForensicCase()	Destructor: delete[] events and delete[] alerts

2.6 Static Members

Two static data members track global program state across all instances:

- Analyst::totalAnalysts — incremented in every constructor, decremented in destructor, read via getTotal()
- ForensicCase::caseCounter — auto-increments on every case creation; assigned to const caseID

2.7 Dynamic Memory Management

Dynamic memory is used throughout to support variable-length data without fixed-size arrays at the class level:

Location	Allocation	Deallocation
Analyst::name	new char[strlen(n)+1] in constructors	delete[] name in destructor
ForensicCase::events	new RawEvent[capacity] — doubles when full	delete[] events in destructor
ForensicCase::alerts	new Alert[alertCapacity] — doubles when full	delete[] alerts in destructor
LogParser* in main()	new SampleLogParser() or new RealLinuxLogParser()	delete parser after use
ForensicCase* in main()	new ForensicCase(...) stored in cases[] array	delete cases[i] on delete/logout

2.8 Operator Overloading / Const Correctness

All getter methods and the report-printing function are marked const, ensuring they cannot modify the object state when called on a const reference. The caseID member is declared const and can only be initialized via the member-initializer list, enforcing immutability after construction.

3. Data Structures

3.1 Struct: RawEvent

Represents a single parsed log entry. Uses char arrays to keep memory layout predictable and compatible with C-style string functions (strcmp, strstr, strcpy):

```
struct RawEvent {
    char timestamp[20];   // e.g. '2026-05-23 02:30:01'
    char eventType[20];   // LOGIN_FAILED, CMD_EXEC, etc.
    char username[30];    // actor identity
    char detail[100];     // event-specific detail
    int  severity;        // 1 = low, 3 = high
    int  hour;            // extracted hour for time-window rules
};
```

3.2 Struct: Alert

Generated by the analysis engine when a threat rule fires:

```
struct Alert {
    char type[40];         // e.g. BRUTE_FORCE
    char description[200];  // human-readable explanation
    char severityTier[10];  // CRITICAL or WARNING
    char timestamp[20];     // time of first triggering event
};
```

3.3 Union: EventSource

Demonstrates C++ union usage to store either an IP address or a hostname for a case source — only one is active at a time, saving memory:

```
union EventSource {
    char ipAddress[32];
    char hostname[32];
};
```

The ForensicCase class stores a bool sourceIsIP alongside the union to track which member is active, following safe union usage practice.

3.4 User Account System

User accounts are stored in a plain struct and managed in a stack-allocated array of size 100. Four hardcoded admin entries are loaded on startup from compile-time constants, then additional users are read from users.txt. The file uses comma-separated values (username, password, role, locked flag):

```
struct UserAccount {
    string username;
    string passwordHash;  // plain-text in current implementation
    string role;          // 'admin' or 'user'
    bool   locked;
};
```

4. Threat Detection Engine

The runAnalysis() method implements 8 independent detection rules. Each rule scans the loaded events array and raises Alert objects if conditions are met. The rules operate in priority order and include de-duplication logic to avoid raising the same alert twice.

Rule #	Alert Type	Tier	Trigger Condition
1	BRUTE_FORCE	CRITICAL	3 or more LOGIN_FAILED events for the same username
2	SUCCESSFUL_BRUTE_FORCE	CRITICAL	LOGIN_SUCCESS preceded by 3+ LOGIN_FAILED for same user
3	SENSITIVE_FILE_ACCESS	CRITICAL	FILE_ACCESS event targeting /etc/passwd, /etc/shadow, SAM, or id_rsa
4	REPEATED_FILE_ACCESS	WARNING	Same file accessed 3 or more times across all FILE_ACCESS events
5	SUSPICIOUS_COMMAND	WARNING	CMD_EXEC event containing whoami, mimikatz, powershell, curl, or net_user
6	PRIVILEGE_ESCALATION	CRITICAL	Any PRIV_ESCALATION event type in the log
7	SUSPICIOUS_LOGIN_TIME	WARNING	LOGIN_SUCCESS or PRIV_ESCALATION between 00:00 and 04:59
8	PORT_SCAN	WARNING	5 or more PORT_SCAN events detected

4.1 Verdict Calculation

After analysis, a verdict is computed from the alert distribution:

Condition	Verdict
2 or more CRITICAL alerts	HIGH RISK
1 CRITICAL, or 3+ WARNING alerts	MEDIUM RISK
1 or more WARNING alerts	LOW RISK
No alerts raised	CLEAN

5. Authentication and Access Control

5.1 Login Flow

- User enters username; system checks existence and locked status first
- Up to 5 password attempts allowed before account is automatically locked
- Core admin accounts (hardcoded) cannot be locked, deleted, or demoted
- Password comparison uses direct plain-text equality (hashPassword() is a pass-through in this implementation)

5.2 Role-Based Menu Access

Once logged in, menu options are filtered by role:

Feature	Admin	User
Create New Case	Yes	No
Load Log File	Yes	Yes
Run Analysis	Yes	Yes
Print / Save Report	Yes	Yes
Generate Sample Log	Yes	No
User Management Panel	Yes	No
Switch / View Cases	Yes	Yes
Delete a Case	Yes	Yes
Deselect Active Case	Yes	Yes

6. File I/O

File	Purpose	Format
users.txt	Persists non-admin user accounts across sessions	CSV: username,password,role,locked(0/1)
<name>.log	Input log file for analysis	Sample format or Linux auth.log syslog format
<report>.txt	Output forensics investigation report	Formatted plain-text case report

The parser auto-selects between SampleLogParser and RealLinuxLogParser based on whether the filename contains 'Linux' or 'log'. This allows transparent handling of both synthetic and real-world log sources without user configuration.

```
May 31 17:36:55 ubuntu sshd[1024]: Failed password for root from 192.168.1.50 port 22 ssh2
May 31 17:36:55 ubuntu sshd[1024]: Failed password for root from 192.168.1.50 port 22 ssh2
May 31 17:36:55 ubuntu sshd[1024]: Failed password for root from 192.168.1.50 port 22 ssh2
May 31 17:36:55 ubuntu sshd[1024]: session opened for user root by (uid=0)
May 31 17:36:55 ubuntu firewall: PORT_SCAN intrusion detected
May 31 17:36:55 ubuntu firewall: PORT_SCAN intrusion detected
May 31 17:36:55 ubuntu firewall: PORT_SCAN intrusion detected
May 31 17:36:55 ubuntu firewall: PORT_SCAN intrusion detected
May 31 17:36:55 ubuntu firewall: PORT_SCAN intrusion detected
May 31 17:36:55 ubuntu sudo:  analyst : TTY=pts/0 ; COMMAND=whoami
May 31 17:36:55 ubuntu sudo:  analyst : TTY=pts/0 ; COMMAND=mimikatz
May 31 17:36:55 ubuntu sudo:  analyst : TTY=pts/0 ; COMMAND=powershell
May 31 17:36:55 ubuntu sudo:  analyst : TTY=pts/0 ; COMMAND=curl
May 31 17:36:55 ubuntu kernel: FILE_ACCESS path=/etc/passwd process=nano
May 31 17:36:55 ubuntu kernel: FILE_ACCESS path=/etc/shadow process=cat
May 31 17:36:55 ubuntu kernel: FILE_ACCESS path=id_rsa process=scp
May 31 17:36:55 ubuntu kernel: FILE_ACCESS path=/etc/passwd process=nano
May 31 17:36:55 ubuntu kernel: FILE_ACCESS path=/etc/passwd process=nano
May 31 17:36:55 ubuntu su: session opened for user root PRIV_ESCALATION
May 23 02:15:00 ubuntu sshd[1025]: session opened for user abdullah by (uid=0)
```

Figure 6.1 Sample log file

7. Sample Report Output

The following is a representative output from printReport() after running analysis on the built-in sample log:

```
======================================================================
                  DIGITAL FORENSICS CASE REPORT
======================================================================
 Case ID      : 2
 Title        : Sample Linux Log File
 Source       : 192.111.222.333
 Investigator : abdullah (Badge #2500705)
----------------------------------------------------------------------
 Events Parsed : 20
 Alerts Raised : 16
----------------------------------------------------------------------
 [1] BRUTE_FORCE [CRITICAL]
     Threshold breached: Total 3 auth failures detected for user: root
     Time: May 31 17:36:55

 [2] SUCCESSFUL_BRUTE_FORCE [CRITICAL]
     Breach hazard: Success login after 3 failures for user: root
     Time: May 31 17:36:55

 [3] SUCCESSFUL_BRUTE_FORCE [CRITICAL]
     Breach hazard: Success login after 3 failures for user: root
     Time: May 31 17:36:55

 [4] SUCCESSFUL_BRUTE_FORCE [CRITICAL]
     Breach hazard: Success login after 3 failures for user: abdullah
     Time: May 23 02:15:00

 [5] SENSITIVE_FILE_ACCESS [CRITICAL]
     Sensitive file accessed: /etc/passwd
     Time: May 31 17:36:55

 [6] SENSITIVE_FILE_ACCESS [CRITICAL]
     Sensitive file accessed: /etc/shadow
```

```
 [10] REPEATED_FILE_ACCESS [WARNING]
     Same file accessed 3+ times: /etc/passwd
     Time: May 31 17:36:55

 [11] SUSPICIOUS_COMMAND [WARNING]
     Suspicious command executed: whoami
     Time: May 31 17:36:55

 [12] SUSPICIOUS_COMMAND [WARNING]
     Suspicious command executed: mimikatz
     Time: May 31 17:36:55

 [13] SUSPICIOUS_COMMAND [WARNING]
     Suspicious command executed: powershell
     Time: May 31 17:36:55

 [14] SUSPICIOUS_COMMAND [WARNING]
     Suspicious command executed: curl
     Time: May 31 17:36:55

 [15] SUSPICIOUS_LOGIN_TIME [WARNING]
     Login activity in anomaly window (00:00 - 05:00) for user: abdullah
     Time: May 23 02:15:00

 [16] PORT_SCAN [WARNING]
     Port scan detected across multiple targets.
     Time: May 31 17:36:55

----------------------------------------------------------------------
 VERDICT : HIGH RISK
======================================================================
```

8. Design Decisions and Limitations

8.1 Design Decisions

- Dynamic arrays with doubling strategy were chosen over std::vector to meet the manual memory management requirement of the OOP course
- The union EventSource demonstrates the C++ union feature even though both members are the same size , it correctly models the semantic concept of 'either IP or hostname'
- The LogParser hierarchy uses pure virtual functions, forcing all derived parsers to implement parseAndLoad() , this enforces the interface contract
- Static caseCounter as a class member ensures unique, sequential IDs without requiring external bookkeeping
- Core admin credentials are hardcoded at compile time so the system always has at least one valid admin even if users.txt is absent or corrupted

8.2 Known Limitations

- Passwords are stored and compared in plain text. A production system would use a cryptographic hash (SHA-256 or bcrypt)
- The maximum case capacity is hardcoded at 10 could be made dynamic with a resizable array or std::vector
- The RealLinuxLogParser only handles a subset of Linux auth.log patterns; coverage of all syslog formats would require a more comprehensive parser
- File I/O for user persistence does not use encryption . users.txt is readable as plain text
- The brute-force rule counts all failures globally rather than within a time window this could produce false positives over long log files

9. Conclusion

The Digital Forensics Log Analyzer successfully demonstrates the four pillars of Object-Oriented Programming encapsulation, inheritance, polymorphism, and abstraction in a practical, real-world application. The project goes beyond a toy example: it implements a working multi-user authentication system, a pluggable log-parsing architecture, a multi-rule threat detection engine, and dynamic memory management with proper cleanup.

The codebase illustrates how OOP concepts reduce code duplication (the LogParser hierarchy), improve maintainability (encapsulated ForensicCase internals), and produce extensible designs (adding a new parser requires only a new derived class, with no changes to client code).The project provides a solid foundation that could be extended with network sockets for live log ingestion, a cryptographic hashing library, a database backend for persistent case storage, and a graphical user interface.